Honeypot IDS – Pool test report

Generated: 2026-10-05 09:26:49
Target: ids (local) (https://localhost)
Exploit sent from each window: CVE-2023-28121 — WooCommerce Payments admin bypass
Windows: 3

Result

INCOMPLETE **Not every frame reached a honeypot pool**

- No session cookie yet: Attacker A, Attacker B, Attacker C (open the shop in that frame first).

What is being tested

Attackers are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees three separate attackers. By default every malicious session is given a honeypot pool **of its own**: the router hands a new attacker a free ready pool no other session owns, and pool_manager builds a spare after each assignment. Two attackers only share a pool when none is free – growth is capped because the host is short of memory, CPU or disk, or every ready pool is already owned – and the router then reuses the ready pools in strict round-robin order.

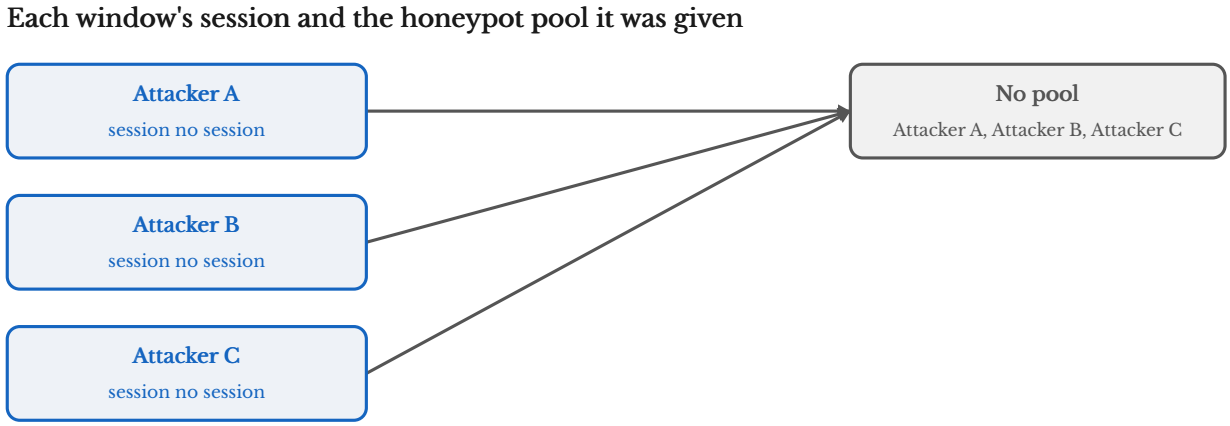

Figure 1. One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

Windows

Window	Session	Honeypot pool	Language	User-Agent (tail)
Attacker A	no session	none	en-US,en;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A
Attacker B	no session	none	de-DE,de;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/B
Attacker C	no session	none	fr-FR,fr;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C

Pool registry at export time

Growing normally – the host has room, so every new attacker gets a pool of their own and a spare is built after each assignment.
Ready pools: 3 (1, 2, 3). Free (unowned) spares: 3. Attackers per pool: pool 1: 0, pool 2: 0, pool 3: 0. Round-robin reuse assignments so far: 0.

What each window showed

Attacker A (no pool) –

Attacker B (no pool) –

Attacker C (no pool) –